Unit 5(Part 1)

Concurrency Control

- Concurrency control is a mechanism in DBMS that allows simultaneous execution of transactions while maintaining data consistency and integrity.
- It ensures that transactions follow ACID properties (Atomicity, Consistency, Isolation, Durability).

Need of Concurrency Control

Concurrency control is required to:

- Prevent conflicts between transactions
- Maintain data consistency
- Ensure correct results in multi-user environment
- Avoid anomalies like dirty read, lost update

Without control: Data may become incorrect or inconsistent

With control: DBMS ensures safe and ordered execution

Problems in Concurrent Execution (Anomalies)

(i) Dirty Read

- Reading *uncommitted data* of another transaction
- If that transaction rolls back → incorrect result

(ii) Lost Update

- Two transactions *update same data*
- One update is overwritten

(iii) Inconsistent Read

- *Same data read multiple times* → different values

Goals of Concurrency Control

- Ensure Isolation of transactions

- Maintain Consistency of database
- Achieve Serializability
- Prevent conflicts and anomalies

Concurrency Control Protocols

Protocols define rules for execution of transactions.

Main Types:

(i) Lock-Based Protocol

(ii) Timestamp-Based Protocol

Types of Schedules Based on Recoverability

- Recoverability ensures that if a transaction fails, the database can be restored to a consistent state.
- It controls when transactions are allowed to commit.
- *Dependency between transactions*

(i) Recoverable Schedule

- A schedule is recoverable if a transaction commits only after the transaction from which it has read commits.

Condition:

If T2 reads data written by T1 → Commit(T1) must happen before Commit(T2)

Example:

T1: W(A)

T2: R(A)

T1: Commit

T2: Commit

Safe because: T2 commits after T1

Advantage: Prevents inconsistency during rollback

(ii) Non-Recoverable Schedule

- A schedule where a transaction commits before the transaction it depends on.

Example:

T1: W(A)

T2: R(A)

T2: Commit

T1: Abort

Problem: T2 already committed using invalid data. Cannot rollback committed transaction → **inconsistent DB**

(iii) Cascading Rollback Schedule

- *A schedule where failure of one transaction causes rollback of other dependent transactions.*

Example:

T1: W(A)

T2: R(A)

T3: R(A)

T1: Abort

Problem:

- T2 and T3 must also rollback
- Leads to multiple rollbacks

Issue:

- High overhead
- Performance degradation

(iv) Cascadeless Schedule

- A schedule in which transactions read only committed data.

Condition: No transaction reads uncommitted data

Example:

T1: W(A), C

T2: R(A), C

Advantages: No cascading rollback and More stable

(v) Strict Schedule

- A schedule where a transaction can read or write a data item only after the previous transaction commits/aborts

Condition: No access to uncommitted data (both read & write restricted)

Example:

T1: W(A), C

T2: R(A), W(A), C

Advantages:

- No dirty read
- No cascading rollback
- Easy recovery

Relationship Between Them

Strict ⊂ Cascadeless ⊂ Recoverable

- Every Strict schedule is also cascadeless & recoverable
- Every Cascadeless schedule is recoverable
- But reverse is NOT true

Advantages of Concurrency Control

- Improves system performance
- Reduces waiting time

- Better resource utilization
- Increases throughput

Disadvantages

- Overhead of managing locks/timestamps
- Deadlock possibility
- Increased complexity
- Reduced concurrency in some cases

LOCK BASED PROTOCOLS

- Lock-based protocol is a concurrency control technique in which a *transaction must acquire a lock before accessing a data item.*
- It ensures that only one transaction can modify data at a time, maintaining consistency.

Purpose

- To prevent data inconsistency
- To ensure isolation of transactions
- To maintain serializability
- To avoid concurrent access conflicts

Types of Locks

(i) Shared Lock (S-lock)

- Used for **read operation**
- Multiple transactions can hold S-lock on the same data item
- Does not allow writing

Example:

- T1 has acquired S(A)
- T2 has requested S(A), Then it is allowed

(ii) Exclusive Lock (X-lock)

- Used for **write operation**
- Only one transaction can hold X-lock
- No other lock (S or X) allowed

Example:

- T1 has acquired X(A)
- T2 has requested S(A), Then it's not allowed

Lock Compatibility

Requested / Existing	Shared(S)	Exclusive(X)
Shared(S)	Yes	No
Exclusive(X)	No	No

- If locks are **compatible → granted**
- If not → transaction **waits**

Basic Locking Rules

- Before Read → acquire S-lock
- Before Write → acquire X-lock
- Lock must be released after use
- If lock not available, then transaction is blocked

Lock-Based Protocols

1. Two-Phase Locking (2PL)

- Divides execution into **two phases**

(i) Growing Phase

- Locks can be acquired
- Locks cannot be released

(ii) Shrinking Phase

- Locks can be released
- No new locks can be acquired

This mechanism, ensures *conflict serializability*

Lock Point

The lock point in a transaction is the moment when the transaction finishes acquiring all the locks it needs.

After this point, no new locks can be added, and the transaction starts releasing locks.

It's a key step in the Two-Phase Locking Protocol to ensure the rules of growing and shrinking phases are followed.

Let's see a transaction implementing 2-PL.

	T1	T2
1	lock-S(A)	
2		lock-S(A)
3	lock-X(B)	

	T1	T2
4	.....	.....
5	Unlock(A)	
6		Lock-X(C)
7	Unlock(B)	
8		Unlock(A)
9		Unlock(C)
10	.....	.....

This is a basic outline of a transaction that demonstrates how locking and unlocking work in the Two-Phase Locking Protocol (2PL).

Transaction T_1

- The growing Phase is from steps 1-3
- The shrinking Phase is from steps 5-7
- Lock Point at 3

Transaction T_2

- The growing Phase is from steps 2-6
- The shrinking Phase is from steps 8-9
- Lock Point at 6

Lock Conversions

In the Two-Phase Locking Protocol, lock conversion means changing the type of lock on data while a transaction is happening.

This process is carefully controlled to maintain consistency in the database.

1. **Upgrading a Lock**:

   This means changing a shared lock (S) to an exclusive lock (X).
   For example, if a transaction initially only needs to read data (S) but later decides it needs to update the same data, it can request an upgrade to an exclusive lock (X).

   However, this can only happen during the Growing Phase, where the transaction is still acquiring locks.

   Example: A transaction reads a value (S lock) but then realizes it needs to modify the value. It upgrades to an X lock during the Growing Phase.

2. **Downgrading a Lock**:

   This means changing an exclusive lock (X) to a shared lock (S).
   For instance, if a transaction initially planned to modify data (X lock) but later decides it only needs to read it, it can downgrade the lock.

   However, this must happen during the Shrinking Phase, where the transaction is releasing locks.
   Example: A transaction modifies a value (X lock) but later only needs to read the value, so it downgrades to an S lock during the Shrinking Phase.

Drawbacks of 2PL

Two-phase locking (2PL) ensures that transactions are executed in the correct order by using two phases: acquiring and releasing locks. However, it has some drawbacks:

- **Deadlocks**: Transactions can get stuck waiting for each other's locks, causing them to freeze indefinitely.
- **Cascading Rollbacks**: If one transaction fails, others that depend on it might also fail leading to inefficiency and potential data issues.

- **Lock Contention**: Too many transactions competing for the same locks can slow down the system, especially when many users are working at the same time.
- **Limited Concurrency**: The strict rules of 2PL can reduce how many transactions can run at once, resulting in slower performance and longer wait times.

Cascading Rollback Problem in 2PL

- In basic Two-Phase Locking (2PL), a transaction can release a lock before it commits.
- *This can cause another transaction to read uncommitted (dirty) data* If the first transaction fails → all dependent transactions must also rollback
- This is called cascading rollback

Simple Example

Transactions:

Transaction T1:

- write A
- unlock A (before commit)

Transaction T2:

- read A

Execution Steps

Step 1: T1 writes A
Step 2: T1 releases (unlocks) A before committing
Step 3: T2 reads A (this is uncommitted data)
Step 4: T1 aborts (fails)

What happens now?

- T2 has read incorrect data
- So T2 must also rollback

Final result: T1 rollback, T2 rollback

*This chain reaction is called **Cascading Rollback***

Why does this happen?

Because in basic 2PL: Locks can be released before commit. This allows other transactions to access uncommitted data

How to avoid it?

- **Strict 2PL**: Ensure that transactions release their locks only after they commit, preventing other transactions from accessing uncommitted changes.
- **Rigorous 2PL**: Extend strict 2PL to hold both read and write locks until the transaction commits, offering even stronger guarantees.

Strict Two-Phase Locking

- Consider the Partial Schedule:

S.No	T_1	T_2
1	lock-X(B)	
2	read(B)	
3	B:=B-50	
4	write(B)	
5		lock-S(A)
6		read(A)
7		lock-S(B)
8	lock-X(A)	
9	.....	.....

The Strict Two-Phase Locking Protocol is a variation of the Two-Phase Locking (2PL) protocol that adds a specific rule for releasing exclusive (X) locks. In

addition to following the two phases (growing and shrinking), strict 2PL ensures that:

All Exclusive (X) Locks are Held Until the Transaction Commits or Aborts:

- A transaction can acquire locks during the growing phase.
- It cannot release any exclusive locks until it has completed its operations and either committed or aborted.
- This rule prevents other transactions from accessing data modified by an ongoing transaction, ensuring data consistency.

Benefits:

- **Recoverable Schedule:** Ensures that if a transaction fails, no other transactions depend on its uncommitted changes.
- **Cascadeless Schedule:** Prevents cascading rollbacks, as other transactions cannot read uncommitted data.

Advantages:

- Prevents dirty reads
- Avoids cascading rollback

Rigorous Two-Phase Locking (Rigorous 2PL)

Rigorous 2PL is a stricter version of the Two-Phase Locking (2PL) protocol. In this protocol:

- *All locks (both shared and exclusive) are held by a transaction until it either commits or aborts.*
- Locks are only released after the transaction finishes, ensuring that no other transaction can access the locked data until the first transaction is fully complete.

This means that no other transaction can read or write the data being used by the current transaction until it is committed or rolled back. This ensures data consistency and avoids issues like dirty reads or cascading rollbacks.

Conservative Two-Phase Locking (Conservative 2PL)

- Conservative 2PL is a variation of the Two-Phase Locking protocol *designed to prevent deadlocks entirely*. It is also known as the Static-2PL.
- **The key feature of Conservative 2PL is that a transaction acquires all the locks it needs at the very beginning of the transaction before it starts executing.**
- If the transaction cannot acquire all the required locks, it does not proceed and waits until all locks become available.

Advantages of Conservative 2PL:

1. **Deadlock-Free:** Deadlocks are avoided because a transaction either acquires all required locks at once or waits until it can.
2. **Efficient Use of Locks:** Transactions do not hold partial locks while waiting for others, reducing contention.
3. **Consistency and Serializability:** Like all 2PL variants, it ensures consistent and serializable schedules.

Problems in Lock-Based Protocol

(i) Deadlock

- Two or more transactions wait indefinitely for each other

Example:

- T1 holds A, waiting for B
- T2 holds B, waiting for A

(ii) Starvation

- A transaction may never get a lock
- Occurs due to continuous priority to other transactions

Advantages

- Simple and easy to implement
- Ensures data consistency

- Widely used in DBMS

Disadvantages

- Deadlock possibility
- Starvation
- Reduced concurrency due to waiting